

Last.fm recommendation project

Data foundation Team handover

Prepared 15 September 2026 | Design Experience | Milestone: acquisition, ingestion and confidence matrices

Status: Both source datasets have been acquired, ingested and converted to sparse preference artifacts. The Python pipeline and Go scaffold have passed automated checks. This is a working data foundation; recommendation training and ranking remain next-stage work.

Delivered	What teammates can use now
Source acquisition	Original HTTPS archives, extracted TSVs, source READMEs and SHA-256 manifests.
DuckDB + Parquet	A persistent catalog over cleaned snapshots; 1K events partitioned by UTC year/month.
Training artifacts	User/item mappings, sparse edge tables and CSR NPZ files for both datasets.
Go + Python repo	Installable Python CLI, locked dependencies, Go confidence CLI and tests.
Handover evidence	Data contract, operations guide, verification report and this PDF.

Why the project exists

The larger proposal extends a movie recommender to music. A discovery stream will suggest unfamiliar tracks; a nostalgia stream will resurface previously favored tracks that have gone dormant but match current listening. This milestone supplies the historical data and implicit-feedback representation needed by both.

Current scale

Metric	1K: tracks	360K: artists
Clean records	19,150,865	17,559,021
Users	992	359,347
Canonical items	1,505,185	268,655
Time partitioning	53 UTC months	Not available in source

Repository: C:/Users/rushi/Desktop/s5/Projects/Design Experience
Start with README.md, then docs/data-contract.md and docs/operations.md.

Data sources and ingestion architecture

Source	Semantics	Important boundary
Last.fm 1K	User, timestamp, artist ID/name, track ID/name.	Individual track plays; supports chronology.
Last.fm 360K	User SHA-1 ID, artist ID/name, total plays.	Artist totals; no listening timestamps.

360K signup dates belong to user profiles and are not listening dates. The two datasets remain separate because their item units and user identifiers differ. Original profiles are retained in raw storage, but demographic tables are not part of the training pipeline. Tags and audio features are not supplied by these archives.

Physical flow

```
MTG HTTPS archives -> data/raw/ + provenance manifests
  1K TSV    -> validate + deduplicate -> year=/month= Parquet
  360K TSV -> validate + aggregate  -> artist-play Parquet
                        |
                    data/lastfm.duckdb
                    recsys.* catalog views
                        |
                    cutoff/counts -> mappings -> sparse matrices
```

Canonical tables

View	Columns / role
recsys.events_1k	user_id, played_at (TIMESTAMPTZ), artist_id, item_id, year, month
recsys.artist_plays_360k	user_id, item_id, play_count (BIGINT)
recsys.edges_1k / edges_360k	Indexed observed pairs, counts, preferences and confidence weights.
recsys.users_* / items_*	Zero-based index to source/canonical ID mappings.

Identity and publication policy

IDs prefer MusicBrainz IDs. Missing artist IDs use a hash of the normalized name; missing track IDs use a hash of the artist identity and track name. Source names and IDs remain in metadata Parquet files. These fallbacks do not solve aliases or merge all name-only records with MBID records.

Each ingestion creates a new snapshot directory. After successful writes, the catalog switches to it. Reruns replace the active view and do not append duplicate data. Old and failed snapshots are retained for audit. Matrix mappings and edge views are published together in a transaction.

Partitioning is implemented as Hive-style Parquet directories queried by DuckDB, not native partitioned DuckDB tables. Include year/month filters plus exact played_at bounds for effective file pruning.

Confidence-weighted preference matrix

A listen is an implicit interest signal, not a star rating. Repetition increases confidence in a positive preference. A missing listen has low-confidence unknown preference; it is not an explicit dislike.

```
n_ui = observed play count for user u and item i
P_ui = 1[n_ui > 0]
C_ui = 1 + kappa * ln(1 + n_ui)
D_ui = C_ui - 1
```

Missing pair: $P_{ui} = 0$, $C_{ui} = 1$, $D_{ui} = 0$

Plays	Preference	Confidence (kappa=40)
0	0	1.000000
1	1	28.725887
9	1	93.103404

The natural logarithm limits the effect of very high play counts. Kappa defaults to 40 and is configurable; it has not been tuned. Kappa is separate from the future discovery/nostalgia mixture parameter alpha.

Temporal contract for 1K

Prepared training cutoff: **2009-05-01 00:00:00 UTC, exclusive**. Only earlier events contribute to counts, indices and last-play times. Active plays are in [cutoff - 90 days, cutoff); historical plays are earlier. Active + historical must equal the total. One ingested user has no pre-cutoff history, so the prepared training matrix has fewer users than the full lake.

Prepared matrix	Users x items	Observed pairs
1K / track	991 x 1,493,688	4,584,444
360K / artist	359,347 x 268,655	17,559,021

Sparse representation and solver boundary

Each snapshot contains edges.parquet, users.parquet, items.parquet and matrix.json. CSR exports are counts.npz, preferences.npz and confidence_delta.npz. Missing sparse entries are zero, so actual confidence is conceptually an all-ones baseline plus the sparse delta. Do not allocate the dense confidence matrix.

Objective for a future implicit factorizer:

```
sum_ui C_ui * (P_ui - dot(X_u, Y_i))^2
+ lambda_reg * (sum_u ||X_u||^2 + sum_i ||Y_i||^2)
```

The objective includes missing pairs with confidence 1. Training only stored positive edges, or taking plain SVD of $C \cdot P$, is different. Confirm the chosen solver's sparse-confidence convention. No factorizer or MSVD-specific regularizers are implemented yet.

Runbook for teammates

Run commands from the repository root. Install Python 3.12+, uv, curl and Go 1.24+. The prepared workspace has a Python 3.12 virtual environment. Go currently uses only its standard library.

Install and verify the scaffold

```
uv sync --frozen --extra dev
uv run pytest -q
go test ./...
go vet ./...
go run ./cmd/recommender -count 9 -kappa 40
```

Acquire and rebuild the data

```
uv run lastfm acquire --dataset 1k
uv run lastfm acquire --dataset 360k
uv run lastfm ingest --dataset 1k
uv run lastfm ingest --dataset 360k
uv run lastfm matrix --dataset 1k \
  --cutoff 2009-05-01T00:00:00Z --npz
uv run lastfm matrix --dataset 360k --npz
uv run lastfm inspect
uv run python scripts/verify.py
```

The backslash above indicates a wrapped command; enter the 1K matrix command on one line in PowerShell. Alternatively use `.venv/Scripts/python.exe -m lastfm.cli` in this Windows workspace. The CLI also accepts `--root` before the subcommand.

Explore the prepared data

```
import duckdb
db = duckdb.connect('data/lastfm.duckdb', read_only=True)
db.execute("SET TimeZone='UTC'")
print(db.sql('SELECT * FROM recsys.edges_1k LIMIT 5'))
```

Operational constraints

Allow at least 15 GB free for downloads, extracted data, Parquet, exports and spill files. DuckDB uses a 2 GB memory limit and four threads; NumPy/SciPy exports require additional process memory. Run writers sequentially. This is not a multi-process serving database.

The DuckDB catalog uses absolute paths to external Parquet. Rebuild views/data after moving the repo; copying the `.duckdb` file alone is insufficient. Archive and derived-data folders are ignored by Git. Teammates can reacquire the data using the CLI.

Acquisition uses HTTPS with certificate verification, retries and resume. Existing manifests are checked against files before reuse. Archive scripts are not executed. SHA-256 values are local integrity fingerprints, not publisher-signed checksums.

Verification results and source anomalies

29 automated verification checks passed on 15 September 2026 at 16:40 UTC. Full evidence is stored in `artifacts/reports/verification.json`; the summary below refers to the prepared full-data run.

Check group	Evidence
Source totals	All original interaction lines accounted for by parsed and quarantined records.
Ingestion conservation	Accepted + validation rejects + merged/duplicate rows reconcile to parsed input.
Preference artifacts	Positive counts, binary preferences, formula agreement and contiguous mappings.
Temporal correctness	No post-cutoff last-play times; active + historical = total; counts match filtered events.
Sparse export	CSR shapes and count sums match the corresponding DuckDB matrix tables.
Partition pruning	EXPLAIN confirms year/month file filters and a reduced file scan.
Code checks	Python integration/boundary tests, Go tests, go vet and cross-language formula smoke test.

Cleaning outcomes

Outcome	1K	360K
Original records	19,150,868	17,559,530
Parser rejects	0	0
Validation rejects	2	88
Collapsed / merged rows	1	421
Clean records	19,150,865	17,559,021

1K contains two out-of-window timestamps and one exact canonical duplicate. Accepted events span 2005-02-14 00:00:07+00:00 to 2009-06-19 21:31:16+00:00.

360K contains 86 rows with date strings in the user-ID field, plus an item without a usable identity and a zero-count row. They are quarantined. Repeated canonical user-artist rows are summed; the report counts how many rows were merged.

Interpretation limits

These checks establish data integrity and pipeline behavior, not recommendation accuracy. There is no trained model, NDCG/Recall result, latency benchmark or demonstrated nostalgia improvement. The source chapter's roughly 83% personalization figure is not a result of this music implementation.

Ownership, next work and references

Workstream	Next concrete deliverable	Acceptance condition
Data / tags	Measure tag coverage and audit identity aliases.	Report missing tags and fallback policy without inventing acoustic data.
Modeling	Implement an implicit-feedback baseline.	Use the confidence contract and save factors with the exact ID mappings.
Evaluation	Define chronological validation and test windows.	Report Recall/NDCG, popularity baseline and cold-start exclusions.
Nostalgia	Retrieve dormant historical favorites.	Use pre-cutoff historical/active counts; compare with simple repeat baselines.
Go service	Load versioned model artifacts and expose retrieval.	Model version, mappings and API contract agree with offline output.

Decisions the team must make

Agree on kappa tuning, validation/test dates, dormant-play thresholds, missing-tag behavior, and how discovery and nostalgia scores will be normalized and mixed. Do not assume a 0.7 mixture weight guarantees seven discovery items in a list of ten. The four-week proposal schedule is a plan, not a measured completion estimate for the remaining work.

Files to read first

README.md - commands and repository map.

docs/data-contract.md - schemas, identity, temporal and confidence semantics.

docs/operations.md - resource limits, reruns, portability and ownership.

docs/verified-state.json - portable result summary.

artifacts/reports/ - full local run evidence.

python/lastfm/ and **internal/preference/** - implementation.

Source and technical references

[UPF / Last.fm 360K dataset description](#)

[Original 1K archive](#)

[Original 360K archive](#)

[DuckDB partitioned writes](#)

Read the README.txt retained with each dataset for provenance and the non-commercial-use condition. The proposal and source chapter supplied in the project root explain the broader research idea. This handover documents the code and data actually prepared in this milestone.

Handoff boundary

The repository is initialized locally. No remote repository or deployment has been created. Raw data is excluded from Git and has not been published. Start the next work from a named matrix snapshot and retain its mappings, cutoff and kappa with all trained artifacts.